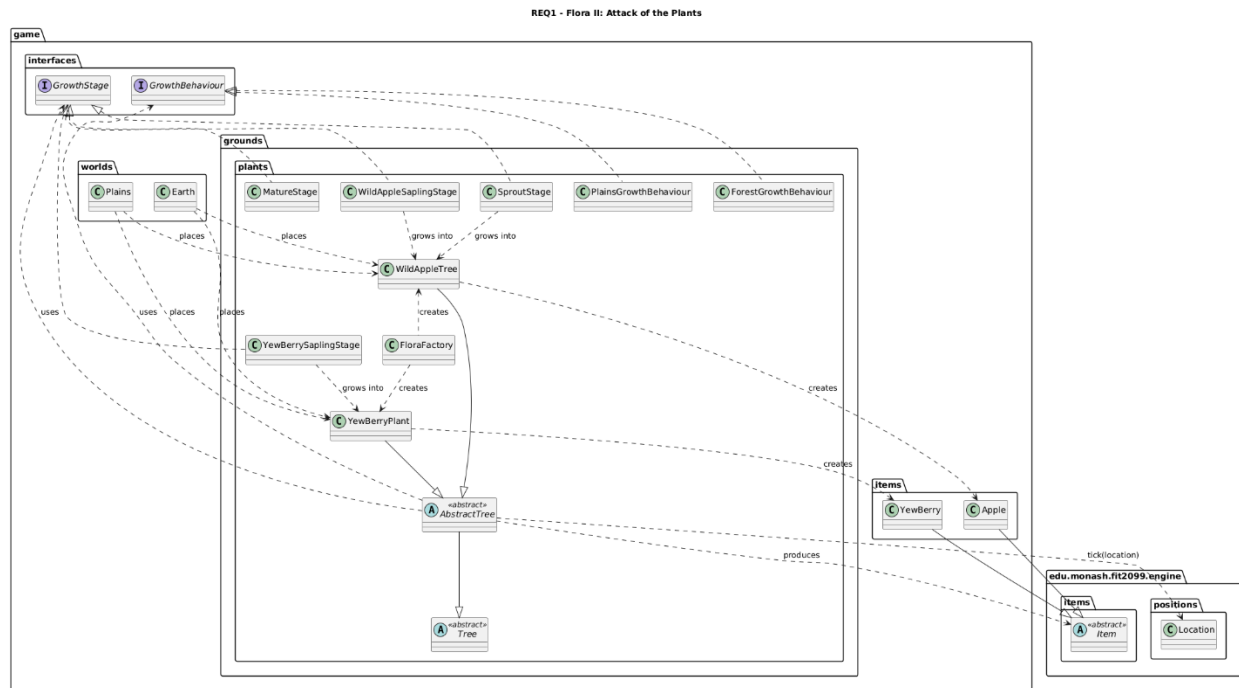




Design Rationale

The design introduces a **dynamic flora growth system** that extends the environmental realism of the world by enabling trees and plants to evolve through multiple growth stages. Each flora instance (e.g., *Wild Apple Tree*, *Yew Berry Plant*) now behaves as a living entity that grows, matures, and produces fruit based on environmental conditions such as forest or plains terrain.

Growth behavior is decomposed into two complementary abstractions:

- **GrowthStage (State Pattern)** – encapsulates lifecycle progression for a plant (e.g., Sprout → Sapling → Mature).
- **GrowthBehaviour (Strategy Pattern)** – defines environment-specific rules (e.g., forest vs. plains growth rates and fruit intervals).

This two-layered abstraction ensures that plant evolution is modular and environment-aware while maintaining independence from the core GameMap logic. The `AbstractTree` class coordinates these patterns, delegating stage management to `GrowthStage` implementations and timing logic to `GrowthBehaviour`. Fruit production remains decoupled and polymorphic through each concrete tree class's `produceFruit()` method, maintaining extensibility and clarity.

The `FloraFactory` centralises flora creation, instantiating trees with the appropriate `GrowthBehaviour` depending on map type. This promotes clean world construction (in Earth and Plains) without duplicating instantiation logic, ensuring easy addition of new flora types or environmental contexts in future requirements.

Advantages of the Design:

- **Single Responsibility Principle (SRP)**
 - Each class performs a distinct function:
 - `AbstractTree` manages growth coordination and fruit timing.
 - `GrowthStage` implementations handle stage transitions and state-specific logic.
 - `GrowthBehaviour` implementations define environment-dependent rates and fruit intervals.
 - `FloraFactory` solely handles flora creation.
- **Open–Closed Principle (OCP)**

- The system is open to extension via new stages or environments (Tundra, etc.) without modifying existing code. This addition of feature does not affect other or previous features of the system.
- **Dependency Inversion Principle (DIP)**
 - AbstractTree depends only on the abstractions (GrowthStage, GrowthBehaviour), not on concrete subclasses. This allows any new flora type or stage to integrate seamlessly through polymorphism, ensuring consistent substitution and inversion of dependencies.
- **Low Coupling and High Cohesion**
 - Stage transitions occur through controlled composition rather than direct manipulation. AbstractTree never checks concrete types (no instanceof), aligning with A2 feedback. Each class encapsulates its data and rules, minimising cross-dependencies and making unit testing straightforward.
- **Extensibility and Reusability**
 - The modular combination of State + Strategy patterns supports infinite combinations of growth and environment types. The FloraFactory enables easy reuse of flora creation logic across maps, and the pattern combination can be reused for other evolving entities (e.g., animals or weather).
- **Connascence**
 - The design mainly demonstrates Connascence of Type (consistent interfaces) and Connascence of Meaning (shared semantics between stages and behaviours). Stronger connascence forms are intentionally avoided by each growth or fruiting rule is self-contained and context-independent and improve modifiability and reducing ripple effects across classes.

Disadvantages of the Design:

- **Stage Duplication Overhead:** Adding a new plant species currently requires creating at least one new stage subclass (e.g., Sprout, Sapling, Mature), which introduces boilerplate despite structural consistency.
- **Potential Complexity in Testing:** Because growth depends on timed updates (tick()), deterministic unit testing requires mocks or time-simulation, adding minor testing complexity.
- **Map Coupling in Factory Use:** While FloraFactory centralises creation, world builders (Earth, Plains) still invoke it manually, creating minimal contextual coupling.

Conclusion

The flora growth system provides a scalable, environment-sensitive model for plant evolution that exemplifies State and Strategy patterns while adhering strongly to SOLID principles. The separation of responsibilities between stage, environment, and fruit production ensures high cohesion, testability, and extensibility. The design's modularity allows effortless addition of new flora types or behaviours in future requirements, maintaining the clarity and maintainability demonstrated in A2's teleportation system while significantly expanding the ecological depth of the FIT2099 world.